Task 9 - Develop APIs for Student Registration, Login, and Course Management.

Backend APIs:

Part 1 — Student registration API

Part 2 — Student login API

Part 3 — Course management APIs

Our API endpoints

These are the endpoints we are going to build.

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Register a student
POST	/api/v1/auth/login	Log in a student
GET	/api/v1/courses	View all courses
GET	/api/v1/courses/:id	View one course
POST	/api/v1/courses	Create a course
PUT	/api/v1/courses/:id	Update a course
DELETE	/api/v1/courses/:id	Delete a course

We will retain our existing course CRUD work and add authentication alongside it.

Prepare the backend folder

Your current backend should look approximately like this:

backend/

├── config/

├── controllers/

│ └── courseController.js

```
├── data/
│   └── courses.js
├── middleware/
│   ├── courseValidation.js
│   ├── errorMiddleware.js
│   └── notFoundMiddleware.js
├── routes/
│   └── courseRoutes.js
├── services/
│   └── courseService.js
├── .env
├── .gitignore
├── package.json
└── server.js
```

We will add these files:

```
backend/
├── controllers/
│   └── authController.js
├── data/
│   └── students.js
├── middleware/
│   └── authValidation.js
├── routes/
│   └── authRoutes.js
└── services/
    └── authService.js
```

Understand the registration workflow

When a student registers, the request travels through these steps:

React Registration Form

Name, email, department, password

|

POST /api/v1/auth/register

Axios sends JSON to Express

|

Validation

Check required fields and email format

|

Controller → Service

Apply registration rules

|

JSON Response

Return student information, never the password

Build Registration and Login APIs

Part 1 — Registration API

Accept student details, validate them, securely hash the password, and return a safe response.

Start here

Part 2 — Login API

Find the student by email and compare the submitted password with its hash.

Part 3 — Test the APIs

Test successful registration, duplicate email, incorrect password, and successful login.

Part 4 — Connect the frontend

Connect your React registration and login forms to the backend.

Step 1 — Open your backend folder

Step 2 — Install the password-hashing package

Run this command inside the backend folder:

npm install bcryptjs

Step 3 — Create the student data file

Inside your existing backend folder, create a data folder if it doesn't already exist. Inside it, create students.js.

Your structure will look like this:

backend/

server.js (existing — keep)

data/

students.js

New

services/

authService.js

New

controllers/

authController.js

New

routes/

authRoutes.js

New

In data/students.js, add:

```
const students = [];

module.exports = students;
```

Step 4 — Create the authentication service

Create services/authService.js.

The service layer contains the main registration and login logic. Keeping it separate from the controller makes the code easier to maintain and test.

```
const bcrypt = require("bcryptjs");
const { randomUUID } = require("crypto");
const students = require("../data/students");

async function registerStudent({ name, email, password, department }) {
 const normalizedEmail = email.trim().toLowerCase();

 const existingStudent = students.find(
  (student) => student.email === normalizedEmail
 );

 if (existingStudent) {
  const error = new Error("Email is already registered");
  error.statusCode = 409;
  throw error;
 }

 const passwordHash = await bcrypt.hash(password, 12);

 const student = {
  id: randomUUID(),
  name: name.trim(),
  email: normalizedEmail,
```

```
    passwordHash,
    department: department.trim(),
    createdAt: new Date().toISOString(),
  };

  students.push(student);

  return {
    id: student.id,
    name: student.name,
    email: student.email,
    department: student.department,
    createdAt: student.createdAt,
  };
}

async function loginStudent({ email, password }) {
  const normalizedEmail = email.trim().toLowerCase();

  const student = students.find(
    (item) => item.email === normalizedEmail
  );

  if (!student) {
    const error = new Error("Invalid email or password");
    error.statusCode = 401;
    throw error;
  }
```

```
  const passwordMatches = await bcrypt.compare(
    password,
    student.passwordHash
  );

  if (!passwordMatches) {
    const error = new Error("Invalid email or password");
    error.statusCode = 401;
    throw error;
  }

  return {
    id: student.id,
    name: student.name,
    email: student.email,
    department: student.department,
  };
}

module.exports = {
  registerStudent,
  loginStudent,
};
```

Step 5 — Create the controller

Create controllers/authController.js.

The controller receives the HTTP request, calls the service, and sends an HTTP response.

```
const authService = require("../services/authService");

async function register(req, res, next) {
 try {
  const student = await authService.registerStudent(req.body);

  return res.status(201).json({
   message: "Student registered successfully",
   student,
  });
 } catch (error) {
  next(error);
 }
}

async function login(req, res, next) {
 try {
  const student = await authService.loginStudent(req.body);

  return res.status(200).json({
   message: "Login successful",
   student,
  });
 } catch (error) {
  next(error);
```

```
 }
}

module.exports = {
 register,
 login,
};
```

Step 6 — Create validation middleware

Create middleware/authValidation.js. If you already have a middleware folder, reuse it.

```
function validateRegistration(req, res, next) {
 const { name, email, password, department } = req.body || {};

 if (
  typeof name !== "string" ||
  typeof email !== "string" ||
  typeof password !== "string" ||
  typeof department !== "string"
 ) {
  return res.status(400).json({
   message: "Name, email, password, and department are required",
  });
 }

 if (
  !name.trim() ||
  !department.trim() ||
```

```
    !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email.trim())
  ) {
    return res.status(400).json({
      message: "Enter a valid name, email, and department",
    });
  }

  if (password.length < 8 || password.length > 72) {
    return res.status(400).json({
      message: "Password must be between 8 and 72 characters",
    });
  }

  next();
}

function validateLogin(req, res, next) {
  const { email, password } = req.body || {};

  if (
    typeof email !== "string" ||
    typeof password !== "string" ||
    !email.trim() ||
    !password ||
    password.length > 72
  ) {
    return res.status(400).json({
      message: "A valid email and password are required",
```

```
  });
 }

 next();
}

module.exports = {
 validateRegistration,
 validateLogin,
};
```

Step 7 — Create the authentication routes

Create routes/authRoutes.js.

```
const express = require("express");
const router = express.Router();

const authController = require("../controllers/authController");

const {
 validateRegistration,
 validateLogin,
} = require("../middleware/authValidation");

router.post(
 "/register",
 validateRegistration,
 authController.register
```

```
);

router.post(
  "/login",
  validateLogin,
  authController.login
);
module.exports = router;
```

Step 8 — Mount the routes without breaking Course CRUD

Open your existing server.js or whichever file creates the Express app.

Find your existing course route mount. It may look like:

```
app.use("/api/v1/courses", courseRoutes);
```

Keep that line exactly as it is. Add the following lines alongside it, adjusting the filename only if your server entry file uses a different name for its route imports:

```
const authRoutes = require("./routes/authRoutes");

app.use(express.json());

// Keep your existing course route mount:
app.use("/api/v1/courses", courseRoutes);

// Add the new authentication route mount:
app.use("/api/v1/auth", authRoutes);
```

Step 9 — Start the server

From the backend folder, run whichever command your package.json supports.

Common options are:

```
npm run dev
```

or:

npm start

Step 10 — Test registration and login

Use Postman, Thunder Client, or another HTTP client. If your backend runs on port 5000, the example URLs below will work; replace 5000 with your actual backend port.

Test A: Register a student

Send a POST request to:

http://localhost:5000/api/v1/auth/register

Choose Body → raw → JSON and enter:


```
{
  "name": "Arun Kumar",
  "email": "arun@example.com",
  "password": "LearnSecure123",
  "department": "Computer Science"
}
```


Expected response: HTTP 201 Created.

Expected response structure

201 Created


```
{
  "message": "Student registered successfully",
  "student": {
    "id": "generated-uuid",
    "name": "Arun Kumar",
    "email": "arun@example.com",
    "department": "Computer Science",
    "createdAt": "generated-timestamp"
```

```
 }
}
```

Test B: Register the same email again

Send the same registration request a second time.

Expected result: HTTP 409 Conflict, with a message that the email is already registered.

Test C: Log in successfully

Send a POST request to:

http://localhost:5000/api/v1/auth/login

Body:

```
{
  "email": "arun@example.com",
  "password": "LearnSecure123"
}
```

Expected result: HTTP 200 OK with the student details and "Login successful".

Test D: Try an incorrect password

Use the same email but change the password, for example to WrongPassword123.

Expected result: HTTP 401 Unauthorized, with "Invalid email or password".

Test E: Verify the existing Course CRUD API

Repeat your existing course API tests:

- GET /api/v1/courses
- GET /api/v1/courses/:id, if implemented
- POST /api/v1/courses
- PUT /api/v1/courses/:id
- DELETE /api/v1/courses/:id

Confirm they still behave exactly as before. If any course route stops working, inspect the server route mounts before changing the course implementation.

Next Step: Connect React to Backend

Step 11 — Registration form

Submit student details to POST /api/v1/auth/register.

Step 12 — Login form

Submit email and password to POST /api/v1/auth/login.

Step 13 — End-to-end testing

Verify success messages, validation errors, and existing course functionality

- Now connect our existing React registration and login pages to your Express authentication APIs using Axios.

The frontend uses ES modules (`import`/`export`) in typical React setups, while your backend remains CommonJS (`require`/`module.exports`). That is perfectly fine; the frontend and backend have separate module systems.

Step 1 — Check Axios and your frontend folder

Open a second terminal in VS Code. Keep the backend terminal running.

Navigate to your React frontend folder. For example, if it is named `frontend`:

```
cd frontend
```

Check whether Axios is installed:

```
npm list axios
```

If Axios is missing, install it:

```
npm install axios
```

If it is already installed, continue without reinstalling.

Step 2 — Create a shared Axios API client

Create this file inside your React application's `src` folder:

```
src/api/apiClient.js
```

If the `api` folder doesn't exist, create it.

```
import axios from "axios";

const apiClient = axios.create({
```

```
  baseURL: import.meta.env.VITE_API_URL ||
"http://localhost:5000/api/v1",
  headers: {
    "Content-Type": "application/json",
  },
});

export default apiClient;
```

Step 3 — Create an authentication service

Create src/api/authApi.js.

This file contains the functions your React pages will call. It keeps HTTP requests separate from the visual form components.

```
import apiClient from "./apiClient";

export async function registerStudent(studentData) {
  const response = await apiClient.post(
    "/auth/register",
    studentData
  );

  return response.data;
}

export async function loginStudent(credentials) {
  const response = await apiClient.post(
    "/auth/login",
    credentials
  );

  return response.data;
}
```

Step 4 — Connect your existing registration page

Open your existing registration component. It might be named Register.jsx, RegisterPage.jsx, or something similar.

Keep your existing JSX and CSS. Add the API integration to your existing component rather than replacing the entire page.

The following is a reference pattern. Adapt the field names and submit handler to match your current form.

```
import { useState } from "react";
import { registerStudent } from "../api/authApi";

function Register() {
 const [formData, setFormData] = useState({
  name: "",
  email: "",
  password: "",
  department: "",
 });
```

```
const [message, setMessage] = useState("");
const [error, setError] = useState("");
const [loading, setLoading] = useState(false);

function handleChange(event) {
 const { name, value } = event.target;

 setFormData((previous) => ({
  ...previous,
  [name]: value,
 }));
}

async function handleSubmit(event) {
 event.preventDefault();

 setMessage("");
 setError("");
 setLoading(true);

 try {
  const result = await registerStudent(formData);

  setMessage(result.message);

  setFormData({
   name: "",
   email: "",
   password: "",
   department: "",
  });
 } catch (err) {
  setError(
   err.response?.data?.message ||
    "Registration failed. Please try again."
  );
 } finally {
  setLoading(false);
 }
}

return (
 <form onSubmit={handleSubmit}>
  <h2>Student Registration</h2>

  <input
   name="name"
```

```
        placeholder="Full name"
        value={formData.name}
        onChange={handleChange}
        required
      />

      <input
        name="email"
        type="email"
        placeholder="Email"
        value={formData.email}
        onChange={handleChange}
        required
      />

      <input
        name="password"
        type="password"
        placeholder="Password (minimum 8 characters)"
        value={formData.password}
        onChange={handleChange}
        minLength={8}
        maxLength={72}
        required
      />

      <input
        name="department"
        placeholder="Department"
        value={formData.department}
        onChange={handleChange}
        required
      />

      <button type="submit" disabled={loading}>
        {loading ? "Registering..." : "Register"}
      </button>

      {message && <p role="status">{message}</p>}
      {error && <p role="alert">{error}</p>}
    </form>
  );
}

export default Register;
```

Step 5 — Connect your existing login page

Open your existing login component. For example, Login.jsx.
Add the import:
import { loginStudent } from "../api/authApi";
Inside your component, add state for the email, password, message, error, and loading indicator:

```
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const [message, setMessage] = useState("");
const [error, setError] = useState("");
const [loading, setLoading] = useState(false);
```

Then add this submit handler:

```
async function handleSubmit(event) {
 event.preventDefault();

 setMessage("");
 setError("");
 setLoading(true);

 try {
  const result = await loginStudent({
   email,
   password,
  });

  setMessage(result.message);

  console.log("Logged-in student:", result.student);

  setPassword("");
 } catch (err) {
  setError(
   err.response?.data?.message ||
    "Login failed. Please try again."
  );
 } finally {
  setLoading(false);
 }
}
```

Connect it to your existing form:

```
<form onSubmit={handleSubmit}>
 <input
  type="email"
  value={email}
  onChange={(event) => setEmail(event.target.value)}
  placeholder="Email"
```

```
  required
 />

 <input
  type="password"
  value={password}
  onChange={(event) => setPassword(event.target.value)}
  placeholder="Password"
  required
 />

 <button type="submit" disabled={loading}>
  {loading ? "Logging in..." : "Login"}
 </button>
```

Step 6 — Test the frontend and backend together

Start the backend and frontend in separate terminals.

Terminal 1 — Backend

Run your existing backend command, for example:

npm run dev

Run this from the backend folder.

Terminal 2 — Frontend

Run:

npm run dev

Run this from the frontend folder. Use your existing start command if different.

Open your application in the browser.

Test these scenarios in order:

Integration test checklist

0/6

Register a new student

Success message and student details

Register the same email again

Duplicate-email error

Log in with the correct password

Login successful message

Log in with the wrong password

Invalid email or password

Open the courses page

Existing course list still loads

Create, update, and delete a test course

Existing Course CRUD still works

Point 9 — Final verification

Completion checklist

Register a new student successfully

Reject duplicate student email

Log in with correct credentials
Reject an incorrect password
Load courses using the existing GET API
Verify existing course create, update, and delete operations

What you have built
Registration API and React form
Student details are validated and passwords are hashed.
Login API and React form
Credentials are checked against the stored password hash.
Existing Course CRUD
Preserved under /api/v1/courses.

One important distinction: student accounts are currently stored in memory, and login does not yet issue a JWT. We'll address protected access next.

Task 10 — JWT Authentication and Protected Routes

Enable the backend to verify who is logged in and restrict access to protected endpoints.
We will implement these stages:

1. Install jsonwebtoken and configure a secret through environment variables.
2. Issue a JWT after successful login.
3. Create authentication middleware to verify tokens.
4. Protect selected API routes without breaking public course browsing.
5. Update Axios to send the token and test authorized versus unauthorized requests.

Point 10: JWT Authentication and Protected Routes, with this access policy:

Action	Access
View courses — GET /api/v1/courses	Public
View one course — GET /api/v1/courses/:id	Public, if your route exists
Create course — POST /api/v1/courses	Login required
Update course — PUT /api/v1/courses/:id	Login required
Delete course — DELETE /api/v1/courses/:id	Login required

We'll keep your CommonJS backend, existing React + Axios frontend, and /api/v1 prefix. We won't replace your working Course CRUD implementation.

How JWT authentication will work

1. Student logs in

Email + password

2. Backend verifies credentials

Compares password with bcrypt hash

3. Backend issues JWT

Signed token with an expiry

4. Axios sends the token

Authorization: Bearer token

5. Middleware verifies JWT

Write operations proceed only with a valid token

Step 1 — Install the JWT package

Open your backend terminal and make sure you are inside the backend folder.

Run:

npm install jsonwebtoken

This package lets Express create and verify JSON Web Tokens.

Check that it is installed:

npm list jsonwebtoken

If your project already loads environment variables with dotenv, keep using that setup. Otherwise, install it:

npm install dotenv

Step 2 — Configure your JWT secret

Open your backend .env file. Add:

JWT_SECRET=replace_this_with_a_long_random_secret

JWT_EXPIRES_IN=1h

Generate a random secret using Node.js:

node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

Copy the generated value into JWT_SECRET.

Important: use the generated secret, not the example placeholder. Never commit .env to Git. Ensure .env is listed in your .gitignore.

Your backend entry file (server.js, for example) must load environment variables before the routes are used. If it doesn't already, add this at the very top:

require("dotenv").config();

Do not add a second copy if it is already there.

Step 3 — Issue a JWT after successful login

Open your existing file:

backend/controllers/authController.js

Find the login function. Replace only that function with the version below. Keep your existing register function unchanged.

```
const jwt = require("jsonwebtoken");
const authService = require("../services/authService");

// Keep your existing register function unchanged.

async function login(req, res, next) {
 try {
  const student = await authService.loginStudent(req.body);

  if (!process.env.JWT_SECRET) {
   throw new Error("JWT_SECRET is not configured");
  }

  const token = jwt.sign(
   {
    email: student.email,
   },
   process.env.JWT_SECRET,
   {
```

```
      subject: student.id,
      expiresIn: process.env.JWT_EXPIRES_IN || "1h",
    }
   );

   return res.status(200).json({
    message: "Login successful",
    token,
    tokenType: "Bearer",
    expiresIn: process.env.JWT_EXPIRES_IN || "1h",
    student,
   });
  } catch (error) {
   next(error);
  }
}

// Preserve your existing register function and export both.
module.exports = {
  register,
  login,
};
```

Step 4 — Create authentication middleware

Create this new file:

backend/middleware/authMiddleware.js

```
const jwt = require("jsonwebtoken");

function authenticateToken(req, res, next) {
 const authHeader = req.headers.authorization;

 if (!authHeader || !authHeader.startsWith("Bearer ")) {
  return res.status(401).json({
   message: "Authentication token is required",
  });
 }

 const token = authHeader.slice("Bearer ".length).trim();

 if (!token) {
  return res.status(401).json({
   message: "Authentication token is required",
  });
 }

 if (!process.env.JWT_SECRET) {
  return res.status(500).json({
   message: "Authentication is not configured",
  });
 }

 try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

```
    req.user = {
      id: decoded.sub,
      email: decoded.email,
    };

    next();
  } catch (error) {
    return res.status(401).json({
      message: "Invalid or expired token",
    });
  }
}

module.exports = {
  authenticateToken,
};
```

Step 5 — Protect write operations in your existing Course CRUD routes

Open your existing course routes file, likely:

backend/routes/courseRoutes.js

At the top, add:

```
const { authenticateToken } = require("../middleware/authMiddleware");
```

Now locate your existing route definitions.

If your routes look similar to these, modify only the middleware argument:

```
router.get("/", courseController.getAllCourses);
```

```
router.get("/:id", courseController.getCourseById);

// Require a valid JWT for changes.
router.post(
  "/",
  authenticateToken,
  courseController.createCourse
);

router.put(
  "/:id",
  authenticateToken,
  courseController.updateCourse
);

router.delete(
  "/:id",
  authenticateToken,
  courseController.deleteCourse
);
```

Keep your existing controller function names. The names above are examples; if your working routes use different names, retain those names and add authenticateToken before the controller.

This preserves public course browsing while requiring authentication for changes.

Public

GET /api/v1/courses

JWT required

POST, PUT, and DELETE /api/v1/courses...

This is authentication only: any valid logged-in student can still modify courses with this configuration. Before production, course management should generally require an admin role. We can add that authorization separately.

Step 6 — Update your Axios client

You already have src/api/apiClient.js. We will extend it so requests can attach the token.

For this initial learning stage, use sessionStorage to keep the token for the current browser tab session. This is a simple demonstration approach, not the strongest production storage design; JavaScript-accessible storage can be exposed by cross-site scripting (XSS).

Replace the contents of src/api/apiClient.js with:

```
import axios from "axios";

const apiClient = axios.create({
  baseURL:
    import.meta.env.VITE_API_URL ||
    "http://localhost:5000/api/v1",
  headers: {
    "Content-Type": "application/json",
  },
});

apiClient.interceptors.request.use((config) => {
  const token = sessionStorage.getItem("token");

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
```

```
  return config;
});

export default apiClient;
```

Step 7 — Save the token after successful login

Open your existing React login page. Inside the successful login section, find the line that calls loginStudent.

Update the success logic so it stores the returned token:

```
const result = await loginStudent({
  email,
  password,
});

sessionStorage.setItem("token", result.token);
setMessage(result.message);
```

Your backend now returns a token field. Without saving it, subsequent Axios requests won't include the JWT.

For logout, add a handler such as:

```
function handleLogout() {
  sessionStorage.removeItem("token");
}
```

Step 8 — Restart and test

Restart the backend after changing the .env file or backend code.

Run your normal backend command, for example:

```
npm run dev
```

Point 10 test checklist

Browse courses without logging in

GET /api/v1/courses should still succeed.

Create a course without a token

POST /api/v1/courses should return 401.

Log in successfully

The response should contain a token.

Create a course with a valid token

Should proceed through authentication.

Send an invalid or expired token

A protected write should return 401.

Verify update and delete

PUT and DELETE require the same valid token.

Activity 1: Install the JWT package only.

Open your backend terminal and run:

npm install jsonwebtoken

Activity 2: Configure the JWT secret

Goal: Keep the secret used to sign and verify JWTs outside your source code.

Step 1: Open your backend .env file

In your backend folder, locate .env. Add these two lines:

JWT_SECRET=your_generated_secret_goes_here

JWT_EXPIRES_IN=1h

Step 2: Generate a secure secret

Run this command in your backend terminal:

node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

Copy the generated value and replace your_generated_secret_goes_here in .env.

Step 3: Check your configuration

- JWT_SECRET holds the secret used to sign and verify tokens.

- JWT_EXPIRES_IN=1h means each token will expire after one hour when we configure token creation.
- Ensure .env is listed in .gitignore so the secret isn't committed to Git.
- If your server.js doesn't already load environment variables, we'll configure that in a later activity.

Important: Don't share your generated secret here or commit it to GitHub.

Activity 3: Load the environment variables

Goal: Make sure your Express backend can read JWT_SECRET from your .env file.

Step 1: Open server.js

At the very top of your existing backend/server.js, check whether this line already exists:

require("dotenv").config();

- If it already exists: Don't change anything.
- If it doesn't exist: Add it at the top of the file.

Step 2: Check whether dotenv is installed

In your backend terminal, run:

npm list dotenv

Activity 4: Generate a JWT after successful login

Goal: When a student logs in successfully, the backend should return a JWT token along with the login response.

Step 1: Open authController.js

Open this file in your backend project:

backend/controllers/authController.js

Find the existing login function. We'll modify only that function and keep your registration code unchanged.

Step 2: Check the imports

At the top of the file, add this line if it isn't already present:

const jwt = require("jsonwebtoken");

Activity 4 — Update authController.js

Step 1: Add the JWT import

Open backend/controllers/authController.js and add this line at the top:

const jwt = require("jsonwebtoken");

Keep your existing imports. Don't duplicate them.

Step 2: Update the login function

Find your existing login function and update it to follow this pattern. This assumes your existing authService.loginStudent() returns a student object with id and email.

```
async function login(req, res, next) {
 try {
  const student = await authService.loginStudent(req.body);

  if (!process.env.JWT_SECRET) {
   throw new Error("JWT_SECRET is not configured");
  }

  const expiresIn = process.env.JWT_EXPIRES_IN || "1h";

  const token = jwt.sign(
   { email: student.email },
   process.env.JWT_SECRET,
   {
    subject: String(student.id),
    expiresIn: expiresIn,
   }
  );

  return res.status(200).json({
```

```
    message: "Login successful",
    token: token,
    tokenType: "Bearer",
    expiresIn: expiresIn,
    student: student,
   });
  } catch (error) {
   next(error);
  }
}
```

Step 3: Save and test

1. Save authController.js.
2. Restart your backend server if it doesn't restart automatically.
3. Log in using a registered student's credentials through your existing frontend or Postman.
4. Check whether the successful response contains token, tokenType, expiresIn, and student.

5. Activity 5 — Verify the token

Goal: Confirm that your backend returns a valid JWT when a student logs in successfully.

Step 1: Start your backend

Open your backend terminal and run your usual server command, for example:

```
npm start
```

Use your project's existing command if it differs.

Step 2: Log in with a registered student

Use your existing login endpoint in Postman, or log in through your React frontend.

If you're using Postman, send a `POST` request to your existing login URL, likely:

```
http://localhost:YOUR_PORT/api/v1/auth/login
```

Replace `YOUR_PORT` with your backend port and adjust the route if yours differs.

Example request body:

```
{
  "email": "student@example.com",
  "password": "your_registered_password"
}
```

Use the credentials of a student you have already registered.

Step 3: Check the response

A successful response should look similar to this:

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "tokenType": "Bearer",
  "expiresIn": "1h",
  "student": {
    "id": "1",
    "email": "student@example.com"
  }
}
```

Activity 5: Test the login response

Start your backend:

npm start

Log in using your existing React frontend or Postman. Your successful login response should contain a token:

{

"message": "Login successful",

"token": "your.jwt.token",

"tokenType": "Bearer",

"expiresIn": "1h",

"student": {

"id": "1",

"email": "student@example.com"

}

}

This is an example response, not a real token. If your login response already returns a token, move to Activity 6.

Activity 6: Create authentication middleware

Goal: Verify the JWT token before allowing access to protected API routes.

Create a new file:

backend/middleware/authMiddleware.js

Add this code:

```
const jwt = require("jsonwebtoken");

function authMiddleware(req, res, next) {
 const authHeader = req.headers.authorization;

 if (!authHeader || !authHeader.startsWith("Bearer ")) {
  return res.status(401).json({
   message: "Authentication token is required"
  });
 }

 const token = authHeader.split(" ")[1];

 try {
  if (!process.env.JWT_SECRET) {
   return res.status(500).json({
    message: "JWT configuration is missing"
   });
  }
```

```
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    req.user = decoded;

    next();
  } catch (error) {
    return res.status(401).json({
      message: "Invalid or expired token"
    });
  }
}

module.exports = authMiddleware;
```

Activity 7: Protect Course API routes

Goal: Allow everyone to view courses, but require authentication to create, update, or delete courses.

Open your existing course routes file, likely:

backend/routes/courseRoutes.js

Import the middleware:

```
const authMiddleware = require("../middleware/authMiddleware");
```

Apply it to your existing POST, PUT, and DELETE routes.

For example, if your current routes use courseController:

```
router.get("/", courseController.getAllCourses);

router.get("/:id", courseController.getCourseById);
```

```
router.post("/", authMiddleware, courseController.createCourse);
router.put("/:id", authMiddleware, courseController.updateCourse);
router.delete("/:id", authMiddleware, courseController.deleteCourse);
```

Activity 8: Attach the token from React

Goal: Automatically send the token with protected API requests.

Step 1: Update the Axios client

Open your existing Axios file, likely src/api/apiClient.js.

Add an Axios request interceptor to your existing client:

```
apiClient.interceptors.request.use(
 (config) => {
  const token = sessionStorage.getItem("token");

  if (token) {
   config.headers = config.headers || {};
   config.headers.Authorization = `Bearer ${token}`;
  }

  return config;
 },
 (error) => Promise.reject(error)
);
```

Step 2: Store the token after login

In your existing login component, after a successful login response, store the token:

```
sessionStorage.setItem("token", response.data.token);
```

For example, if you already have an Axios login request:

const response = await apiClient.post("/auth/login", formData);

sessionStorage.setItem("token", response.data.token);

Step 3: Clear the token on logout

In your existing logout function, add:

sessionStorage.removeItem("token");

sessionStorage is suitable for a learning demo, but JavaScript can access it. For production applications, consider secure, HttpOnly, Secure, appropriately configured cookies and a broader security review.

Activity 9: Test protected endpoints

Use Postman or your React application to verify these cases.

Test	Expected result
View courses without a token	200 OK, if the existing GET route works
Create a course without a token	401 Unauthorized
Update a course without a token	401 Unauthorized
Delete a course without a token	401 Unauthorized
Create a course with a valid token	Allowed, subject to validation
Use an invalid or expired token	401 Unauthorized

Activity 10: Verify the complete flow

Follow this end-to-end test:

1. Register a student

Confirm registration works as before.

2. Log in

Confirm the response contains a JWT token.

3. Send a protected request

Confirm Axios sends the Bearer token and the backend accepts it.

4. Remove the token and retry

Confirm protected write requests are rejected with 401.

Activity 11: Final security checks

- Keep .env out of Git using .gitignore.
- Never share or commit your actual JWT_SECRET.
- Ensure dotenv loads before routes or controllers need environment variables.
- Never put passwords or the JWT secret inside the token.
- Remember that JWT payloads are signed, not encrypted.
- Restarting your backend may reset student accounts if your current registration system uses an in-memory array.